RSpec's solution to this problem couldn't be simpler: a plain Ruby hash. Every example and example group gets its own such hash, known as the *metadata hash*. RSpec populates this hash with any metadata you've explicitly tagged the example with, plus some useful entries of its own.

Metadata Defined By RSpec

A good example is worth a thousand words, so let's jump right into one. In a fresh directory, create a file called metadata_spec.rb with the following contents:

08-metadata/01/spec/metadata_spec.rb

```
require 'pp'

RSpec.describe Hash do
  it 'is used by RSpec for metadata' do |example|
    pp example.metadata
  end
end
```

This snippet shows something we haven't talked about before: getting access to your example's properties at runtime. You can do so by having your it block take an argument. RSpec will pass an object representing the currently running example. We'll revisit this topic later in the chapter.

The call to example.metadata returns a hash containing all the metadata. We're using the pp (short for *pretty-print*) function from Ruby's standard library to dump the contents of the hash in an easy-to-read format.

Go ahead and run the example:

```
$ rspec spec/metadata_spec.rb
{:block=>
  #<Proc:0x007fa6fc07e6a8@~/code/metadata/spec/metadata_spec.rb:4>,
 :description_args=>["is used by RSpec for metadata"],
 :description=>"is used by RSpec for metadata",
 :full_description=>"Hash is used by RSpec for metadata",
 :described_class=>Hash,
 :file_path=>"./spec/metadata_spec.rb",
 :line_number=>4,
 :location=>"./spec/metadata_spec.rb:4",
 :absolute_file_path=>
  "~/code/metadata/spec/metadata_spec.rb",
 :rerun_file_path=>"./spec/metadata_spec.rb",
 :scoped_id=>"1:1",
 :execution_result=>
  #<RSpec::Core::Example::ExecutionResult:0x007ffda2846a78
   @started_at=2017-06-13 13:34:00 -0700>,
 :example_group=>
  {:block=>
    #<Proc:0x007fa6fb914bb0@~/code/metadata/spec/metadata_spec.rb:3>,
```